Milestone-1

By-Akhil B Raj

USN-4SF24CS009

TOPIC- SAAS DOMAIN

Given Schema

```
UserSubscriptions (
  user_id,
  user_name,
  user_email,
  organization_name,
  role,
  plan_name,
  plan_price,
  billing_cycle,
  feature_list,
  usage_limit,
  usage_count,
  start_date,
  end_date,
  payment_status
)
```

A1-IDENTIFYING ISSUES

The given schema is not at all relavant. Entities could be easily categorized into 4 parts-

- User Details
- Organisation Details
- Plan Details
- Payment Details

The above entities could be easily categoried under this following schemas.

Also there will be redundancy in entities like username. There can also be a situation where a company buys multiple service from the SaaS company. Thus there occurs a chance there is redundancy in company details.

Also while breaking down given schema into schema which is useful, we should make sure to avoid partial dependency.Payment related details are usually depended on the purchased plan,while

organisations may choose from multiple plans. Thus while making tables, if we place these 3 in same table, then payment is partially depended on the organisation.

Also, Organisation->Plan->Payment, thus Organisation->Payment also falls under 3NF.

Thus we should keep these following details in mind while making tables and schema.

A2. Normalize the Schema

The UserSubscription Schema can can easily broken down into 4 simplified Schemas-

User(user_id,
  user_name,
  user_email,
Phone number,
  Password,
  organization_name,
  Role)

Organisation(
Organisation_name,
Service Provided,
Number of employees)
Plan(
Plan name,
Number of users allowed,
Plan cost,
Features Available,
duration)

Purchase(
Plan Purchased,
Organisation id,
Bill date,
)

1-Now, lets focus on User Schema,

User(user_id,
  user_name,
  user_email,
  Phone-number
  Password,
  organization_name,
  Role)

There is a chance that user has multiple phone numbers,thus to make the table atomic, we should place phone number and userId in different schema.
The user can only login under company email, thus email is atomic in nature.
Also, there maybe a chance that user has multiple roles.Thus User and roles should have separate schema.
Then it satisfies 1NF.

If we keep only UserId as primary key, then it satisfies 2NF and in this case, also 3NF.
Thus new schema will be -

User(user_id,
  user_name,
  user_email,
  Password,
  organization_name,
)

UserNumber(
UserId,

Ph-no)

UserRole(
UserID,
Role)

2-Organisation(

Organisation_name,

About,

Service Provided,

Number of employees)

This is a probability that a company provides multiple services, thus company-id and service should be in separate table.

Organisation(

Org-id,

Organisation name,

About)

Organisation_Service(

Organisation_id,

Service provided)

The following tables satisfies 1NF,2NF and 3NF

3- Plan(

Plan name,

Number of users allowed,

Plan cost,

Features Available,

duration)

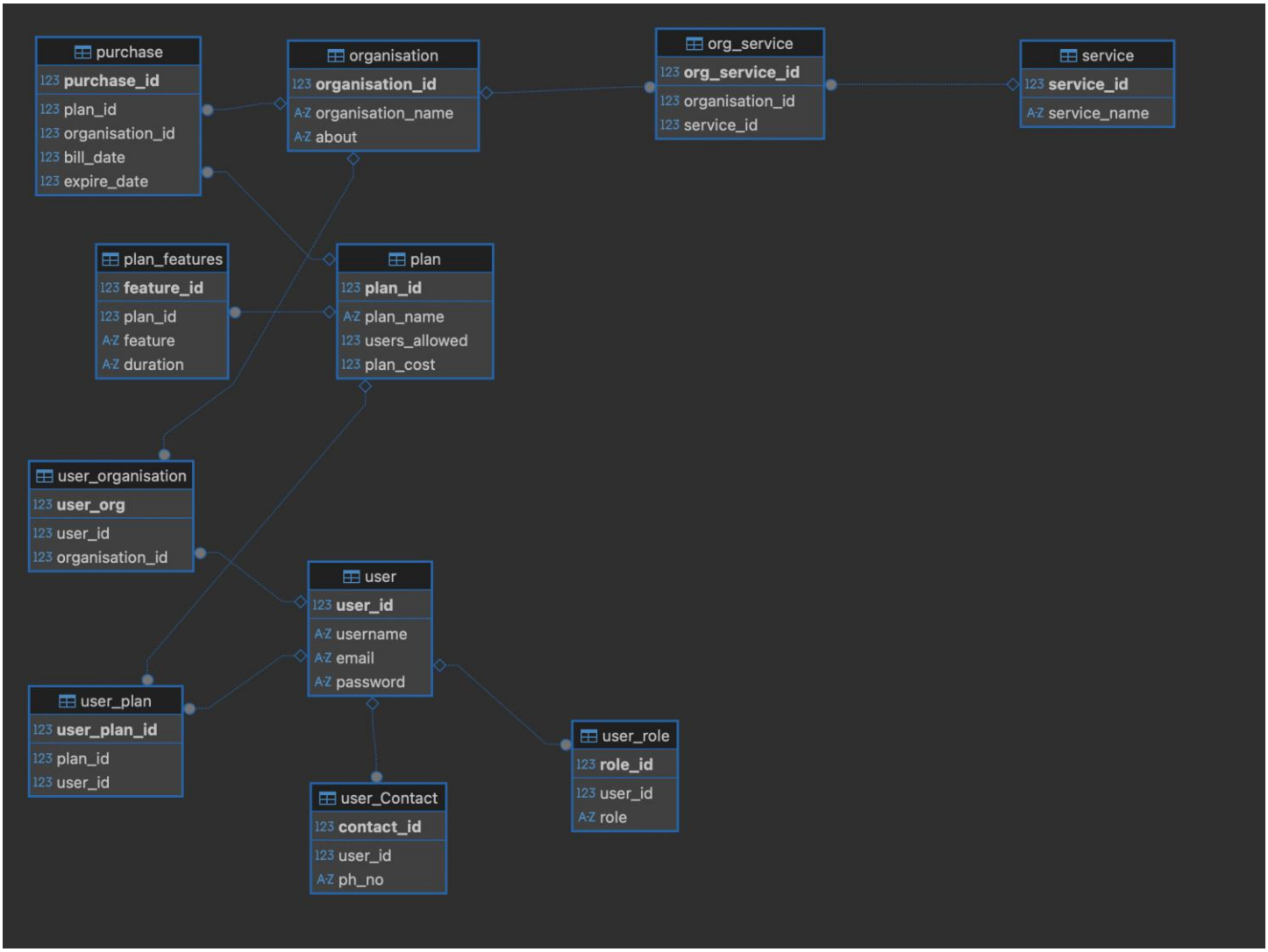
Here multiple features are available thus features should be placed in different table along with plan id to satisfy 1NF.

A3-Assumptions

We have considered the following assumptions while creating this schema-

- User will be alloted with company email id, thus it is atomic in nature
- User can have multiple phone number
- User can be assigned with multiple roles in his organisation
- Organisation may provide different types of services(like Amazon's primary income is through AWS, delivery is secondary)
- A single organisation might buy multiple subscription thus multiple subscription at same time

ER DIAGRAM


purchase
123 purchase_id
123 plan_id
123 organisation_id
123 bill_date
123 expire_date
organisation
123 organisation_id
A-Z organisation_name
A-Z about
org_service
123 org_service_id
123 organisation_id
123 service_id
service
123 service_id
A-Z service_name
plan_features
123 feature_id
123 plan_id
A-Z feature
A-Z duration
plan
123 plan_id
A-Z plan_name
123 users_allowed
123 plan_cost
user_organisation
123 user_org
123 user_id
123 organisation_id
user
123 user_id
A-Z username
A-Z email
A-Z password
user_plan
123 user_plan_id
123 plan_id
123 user_id
user_role
123 role_id
123 user_id
A-Z role
user_Contact
123 contact_id
123 user_id
A-Z ph_no

SECTION B — DATABASE DESIGN FROM REQUIREMENTS

B1-Requirement Clarification

1-If a user is part of multiple organisation,then how to decide which plan does he have access to?

2-Is organisation able to cancle subscription?

3-How are things managed when organisation shifts from one plan to another and how pricing is managed in such a situation?

4-Should useage be calculated in individual level or organisational level?

5-Can users who dont belong to any organisation signup?

B2-Design

```
create table organisation(
organisation_id integer primary key autoincrement,
organisation_name text
);

create table user(
user_id integer primary key autoincrement,
user_name text,
user_email text
);

create table user_contact(
contact_id integer primary key autoincrement,
user_id integer,
```

```
contact_no text,
foreign key (user_id) references user(user_id)
);

create table user_org(
user_org_id integer primary key autoincrement,
user_id integer,
organisation_id integer,
foreign key (user_id) references user(user_id),
foreign key(organisation_id) references organisation(organisation_id)
);

create table role(
role_id integer primary key autoincrement,
role_name text
);

create table user_role(
user_role_id integer primary key autoincrement,
user_id integer,
role_id integer,
foreign key (user_id) references user(user_id),
foreign key (role_id) references role(role_id)
);

create table plan(
plan_id integer primary key autoincrement,
```

```
plan_name text,
price integer
);

create table plan_duration(
duration_id integer primary key autoincrement,
plan_id integer,
duration text,
foreign key (plan_id) references plan(plan_id)
);

create table feature(
feature_id integer primary key autoincrement,
feature text
);

create table plan_feature(
plan_feature_id integer primary key autoincrement,
plan_id integer,
feature_id integer,
foreign key (plan_id) references plan(plan_id),
foreign key (feature_id) references feature(feature_id)
);

create table organisation_plan(
organisation_plan_id integer primary key autoincrement,
organisation_id integer,
```

```
plan_id integer,
starting_date text,
ending_date text,
foreign key (organisation_id) references organisation(organisation_id),
foreign key (plan_id) references plan(plan_id)
);
create table plan_usage(
usage_id integer primary key autoincrement,
plan_id integer,
usage_limit integer,
remaining_usage integer,
foreign key (plan_id) references plan(plan_id)
);

create table payment(
payment_id integer primary key autoincrement,
payment_type text,
payment_amount integer,
organisation_id integer,
plan_id integer,
foreign key (plan_id) references plan(plan_id),
foreign key (organisation_id) references organisation(organisation_id)
);
```

B3- Normalisation

To satisfy N1 normalisation, we need to make sure that the values in table are atomic. To achieve this, we have written User_contact table separately instead of adding it with users.

```
create table user_contact(
contact_id integer primary key autoincrement,
user_id integer,
contact_no text,
foreign key (user_id) references user(user_id)
);
```

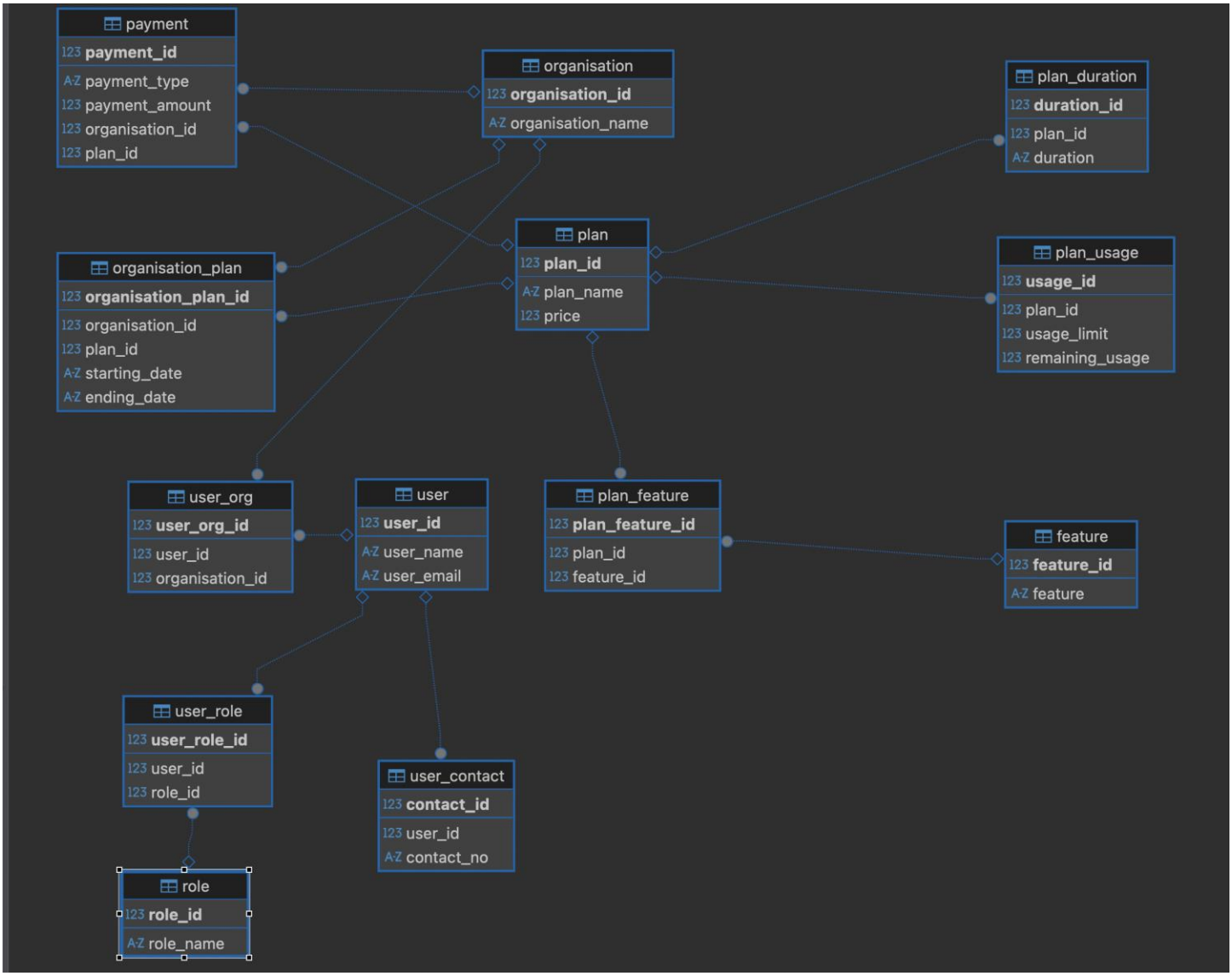
Also, as organisation may have multiple purchases, to make it atomic, we have made the following table-

```
create table organisation_plan(
organisation_plan_id integer primary key autoincrement,
organisation_id integer,
plan_id integer,
starting_date text,
ending_date text,
foreign key (organisation_id) references organisation(organisation_id),
foreign key (plan_id) references plan(plan_id)
);
```

Since we have only 1 primary key for each table, and it satisfies NF1, we can say that each table satisfies NF2.

Also, table is made in such a way that no non primary key is depended on other non-primary key.Thus it satisfies NF3.

ER DIAGRAM



SECTION C — QUERIES AND TRANSACTIONS

Retrieve all organizations a given user belongs to, along with their role in each organization.

sqliteonline.com

Import Export

Run SQLite

```sql
SELECT u.user_name,
       o.organisation_name,
       r.role_name
FROM user u
INNER JOIN user_org uo
ON u.user_id = uo.user_id
INNER JOIN organisation o
ON uo.organisation_id = o.organisation_id
INNER JOIN user_role ur
ON u.user_id = ur.user_id
INNER JOIN role r
ON ur.role_id = r.role_id
WHERE u.user_name = 'akhil':
```

user_name	organisation_name	role_name
akhil	Amazon	HR

2- Retrieve the current active subscription plan for each organization along with its usage details.

sqliteonline.com

Import Export

Run SQLite

```sql
SELECT o.organisation_name,
       p.plan_name,
       p.price,
       pu.usage_limit,
       op.ending_date
FROM organisation o
JOIN organisation_plan op
ON o.organisation_id = op.organisation_id
JOIN PLAN p
ON op.plan_id = p.plan_id
JOIN plan_usage pu
ON p.plan_id = pu.plan_id
```

organisatio...	plan_name	price	usage_limit	ending_date
Amazon	Basic Plan	5000	100	2026-06-30
Amazon	Standard Plan	10000	300	2026-12-31